Aprendizaje automático

Introducción, modelos simples, preprocesamiento y validación

Curso IFCD107 – Especialista en Inteligencia Artificial

Adaptación didáctica al castellano del material de Ryan Miller

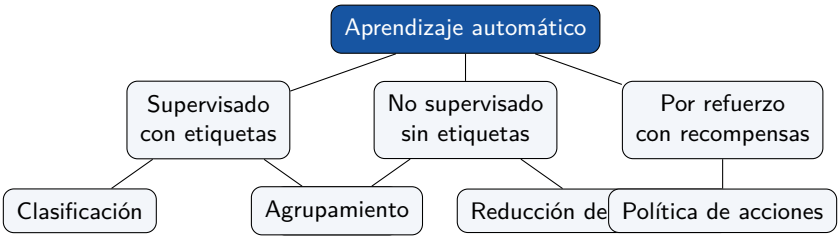
¿Qué es el aprendizaje automático?

- Es una rama de la inteligencia artificial que utiliza datos y algoritmos para aprender patrones a partir de la experiencia.
- El rendimiento puede mejorar al disponer de ejemplos adecuados y un proceso de aprendizaje bien planteado.
- El aprendizaje automático aproxima relaciones útiles; no implica que el sistema razone como una persona.

Definición operativa

Un modelo aprende de los datos una regla que después aplica a casos que aún no ha visto.

Panorama del aprendizaje automático



En aprendizaje supervisado se conocen respuestas objetivo; en el no supervisado se buscan estructuras; en el aprendizaje por refuerzo se aprende mediante interacción y recompensas.

Datos tabulares: filas y columnas

Trabajaremos con una matriz de datos X formada por n observaciones y p características:

$$X = \begin{pmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,p} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n,1} & x_{n,2} & \cdots & x_{n,p} \end{pmatrix}$$

- Cada fila representa una observación; cada columna, una característica.
- Usaremos x_i para el vector de características de la observación i, y x_j para los valores de la característica j.
- Las letras mayúsculas suelen representar matrices y las minúsculas vectores o valores.

Aprendizaje supervisado: entradas y objetivo

En regresión, el objetivo numérico se modela como:

$$y_i = f(x_i) + \varepsilon_i$$

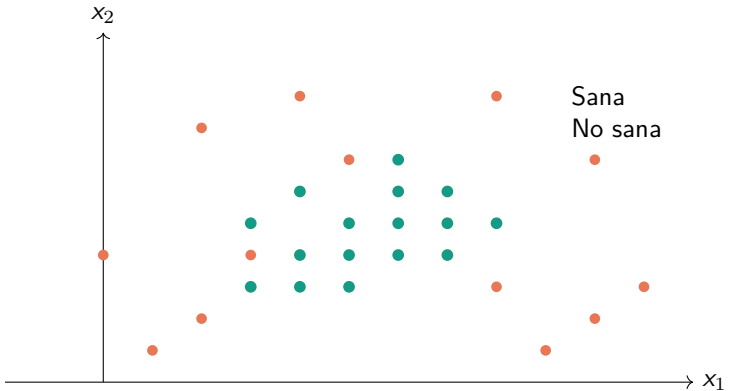
En clasificación, el modelo estima la probabilidad de una clase a partir de las características:

$$\Pr(Y = c \mid X = x_i) = f_c(x_i)$$

- f representa una relación desconocida que queremos aproximar.
- ε_i recoge variación no explicada por las entradas.
- El algoritmo produce una aproximación a partir de los ejemplos disponibles.

Ejemplo de clasificación: muestras sanas

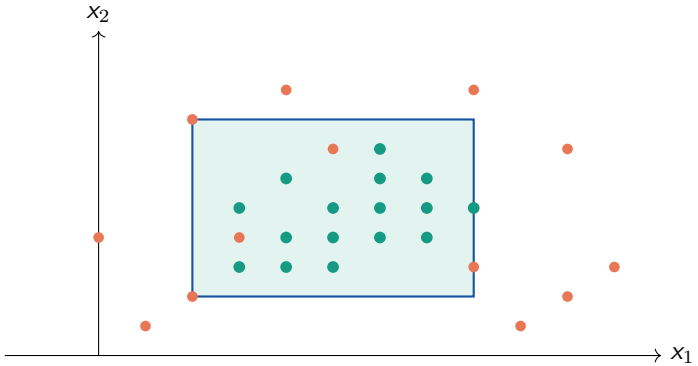
Supongamos dos características, x_1 y x_2, y dos clases: **sana** y **no sana**. ¿Es posible predecir la clase a partir de las dos medidas?



Datos sintéticos esquemáticos: las muestras sanas tienden a agruparse en una región, aunque hay solapamiento.

Una frontera de decisión sencilla

Una regla basada en el rectángulo $2 \leq x_1 \leq 8$ y $2 \leq x_2 \leq 8$ clasifica como sana una observación situada dentro de esa región.



Resultado del ejemplo original

La regla ilustrada clasifica correctamente 178 de 200 observaciones: un 89 % de exactitud.

Error reducible e irreducible

- Algunas muestras sanas quedan fuera de la región y algunas no sanas caen dentro.
- El error irreducible refleja ruido, variables no observadas y solapamiento real entre clases.
- Aunque conociéramos perfectamente la relación subyacente, no necesariamente alcanzaríamos el 100 % de acierto.
- El objetivo práctico es reducir el error que sí depende de una elección mejor de datos, variables o modelo.

Objetivo del aprendizaje automático

Minimizar el error reducible sin confundir el ajuste al entrenamiento con la capacidad de generalizar.

Ejemplo real: asignación de incidencias

El material original describe un proyecto para asignar automáticamente solicitudes de soporte informático a un departamento o persona:

1. Se usan solicitudes históricas y sus destinos como ejemplos.
2. Se entrena un modelo para reproducir patrones de asignación.
3. La asignación humana también puede contener inconsistencias o decisiones subjetivas.

Consecuencia

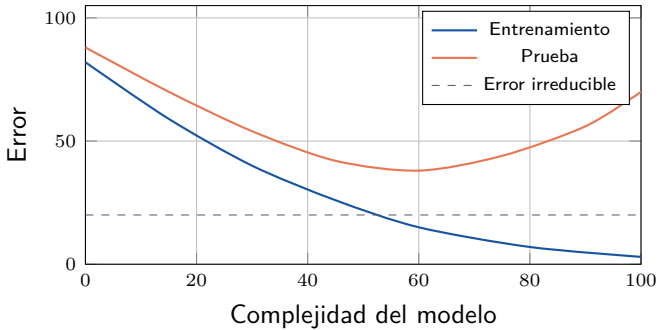
Si las etiquetas históricas contienen ruido, el modelo no puede aprender una regla perfecta solo a partir de ellas.

Entrenamiento y prueba

- No basta con medir el error en los datos ya observados.
- Queremos estimar el rendimiento sobre datos nuevos que el modelo todavía no ha visto.
- Se separan los datos disponibles en conjuntos de entrenamiento y prueba.
- El entrenamiento sirve para ajustar el modelo; la prueba, para evaluar una vez su rendimiento final.



Complejidad del modelo y error



Curvas esquemáticas; representan la relación descrita en el material, no datos medidos.

- Aumentar la complejidad puede reducir el error de entrenamiento.
- El error de prueba puede dejar de mejorar y aumentar por sobreajuste.
- La línea de referencia representa 20 unidades de error irreducible en el ejemplo.

Sesgo y varianza

El error reducible puede deberse a dos fuentes:

Sesgo alto

El modelo es demasiado rígido para captar aspectos importantes de la relación entre variables. Se asocia al **subajuste**.

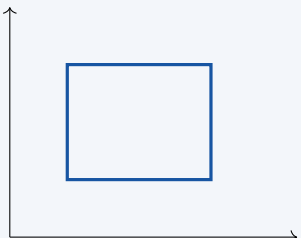
Varianza alta

El modelo reacciona demasiado a peculiaridades de la muestra de entrenamiento. Se asocia al **sobreajuste**.

El reto es equilibrar flexibilidad y estabilidad.

Complejidad de la frontera de decisión

Rectángulo



Una regla sencilla puede tener alto sesgo y baja varianza.

Polígono complejo



Una regla muy flexible puede tener menor sesgo y mayor varianza.

Cómo medir el error

- En clasificación binaria, la exactitud resume la proporción de etiquetas predichas correctamente.
- Para una respuesta numérica, medimos las distancias entre valor observado y predicho.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Aquí y_i es el valor observado y $\hat{y}_i$ la predicción para la observación i. RMSE penaliza más los errores grandes; MAE expresa el error absoluto medio.

Repaso: conceptos de la introducción

1. Diferenciar clasificación y regresión.
2. Explicar error reducible e irreducible con un ejemplo.
3. Describir el equilibrio entre sesgo y varianza.
4. Justificar la separación entre entrenamiento y prueba.

Modelos sencillos para clasificar

Partimos de observaciones descritas por características y etiquetadas como sanas o no sanas. El objetivo es asignar una clase a una nueva observación usando patrones aprendidos.



K vecinos más cercanos (K-NN)

- Para clasificar un nuevo punto, se buscan las observaciones de entrenamiento más próximas.
- Con un vecino, se copia la etiqueta del punto más cercano.
- Con k vecinos, se combinan las etiquetas de los k puntos próximos.
- La noción de proximidad depende de la métrica y de la escala de las características.

Intuición

Puntos con características parecidas suelen recibir predicciones parecidas.

Distancia de Minkowski

La distancia entre dos observaciones a y b, con m características, se define como:

$$d_{a,b} = \left(\sum_{j=1}^{m} |x_{a,j} - x_{b,j}|^p \right)^{1/p}$$

- Suma diferencias coordenada a coordenada.
- $p = 2$ produce la distancia euclídea.
- $p = 1$ produce la distancia Manhattan.
- El parámetro p se selecciona según el problema y se puede tratar como hiperparámetro.

Cómo combinar los vecinos

Ponderación uniforme

Todos los vecinos cuentan igual. Si 4 de los 5 vecinos pertenecen a la clase sana, la probabilidad estimada de esa clase es $4/5 = 80\,\%$.

Ponderación por distancia

Los puntos cercanos tienen más peso, por ejemplo mediante el inverso de la distancia.

En regresión K-NN predice promediando los valores objetivo de los vecinos, en lugar de calcular proporciones de clases.

Hiperparámetros de K-NN

K-NN es útil para estudiar dos ideas generales:

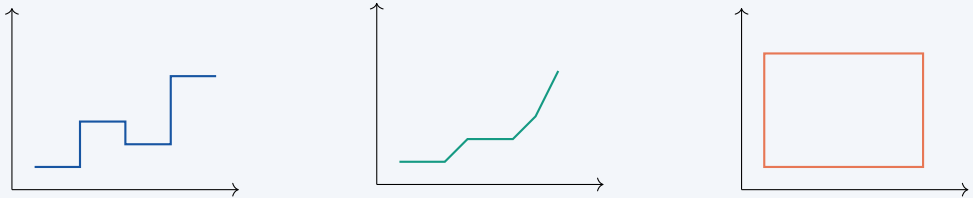
- **Hiperparámetros**: opciones configuradas antes de ajustar el modelo.
- **Preprocesamiento**: transformaciones necesarias para que el algoritmo funcione adecuadamente.

En K-NN, los hiperparámetros principales son:

- k (o `n_neighbors`): número de vecinos.
- p: potencia usada por la distancia de Minkowski.
- `weights`: ponderación uniforme o por distancia.

K-NN: efecto del número de vecinos

$k = 1$	$k = 5$	$k = 35$
Frontera muy irregular	Frontera más suave	Regiones amplias



Esquemas cualitativos de fronteras de decisión; al aumentar k, la predicción suele ser menos sensible a observaciones individuales.

K-NN y sesgo-varianza

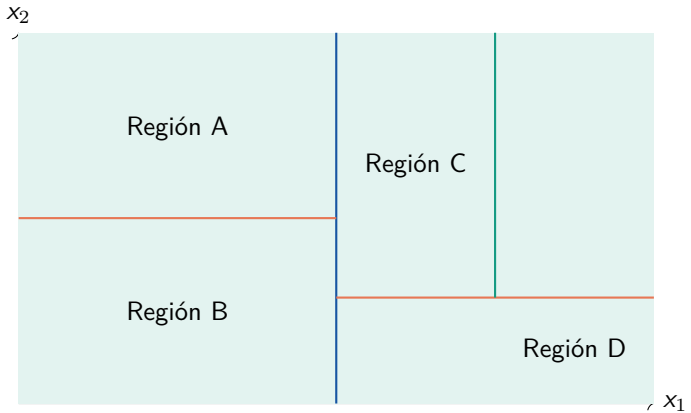
- Valores pequeños de k dan modelos flexibles, con menor sesgo y mayor varianza.
- Valores grandes producen fronteras más suaves, mayor sesgo y menor varianza.
- La ponderación por distancia puede suavizar la frontera frente a la ponderación uniforme.
- La elección final debe basarse en una estimación de generalización, no solo en el ajuste de entrenamiento.

Árboles de decisión

Un árbol divide recursivamente el espacio de características en regiones cada vez más homogéneas:

1. Se empieza por un nodo raíz que contiene los datos.
2. Se busca una regla que divida los datos en nodos hijos más puros.
3. Se repite el proceso en los nodos que todavía pueden dividirse.
4. El crecimiento se detiene al cumplir un criterio de parada.

Las divisiones generan regiones rectangulares



Cada regla binaria crea nuevas regiones alineadas con una característica, como $x_1 < c$ o $x_2 < c$.

Impureza de Gini y error cuadrático

Para clasificación, una medida habitual de impureza es:

$$G = \sum_{j=1}^{K} p_j(1 - p_j) = 1 - \sum_{j=1}^{K} p_j^2$$

En clasificación binaria:

$$G = p_1(1 - p_1) + p_2(1 - p_2)$$

- El algoritmo considera cortes posibles en cada característica y elige el que más reduce la impureza ponderada.
- Los cortes útiles se ubican entre valores observados: mover el corte dentro de un intervalo sin datos no cambia la partición.
- En regresión se suele medir la heterogeneidad con la suma de errores cuadrados: $\sum_i (y_i - \hat{y}_i)^2$.

Ejemplo de ganancia de Gini

Impureza inicial con proporciones 0,42 y 0,58:

$$0{,}42(1-0{,}42)+0{,}58(1-0{,}58)=0{,}487$$

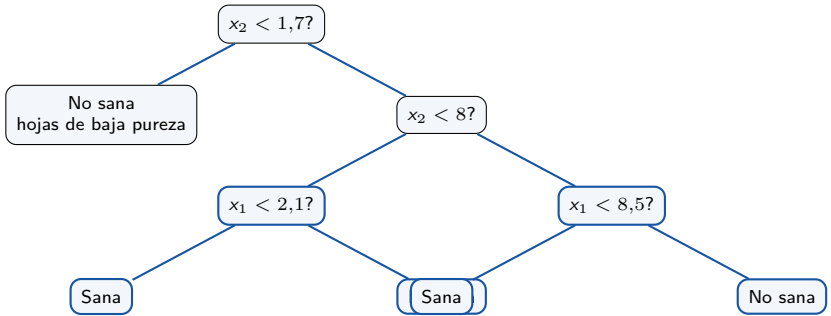
Después de la primera división, el nodo izquierdo (82 %) tiene proporciones 0,52 y 0,48; el nodo derecho (18 %) es puro:

$$0{,}82[0{,}52(1-0{,}52)+0{,}48(1-0{,}48)]+0{,}18[0]=0{,}409$$

Ganancia de Gini

$0{,}487-0{,}409=0{,}078$: la división reduce la impureza ponderada.

Representación de un árbol



Cada nodo interno plantea una regla; cada hoja produce una clase (o un valor numérico). Las proporciones de clase de la hoja pueden interpretarse como probabilidades estimadas.

Controlar el crecimiento del árbol

Un árbol puede crecer hasta ajustar perfectamente el entrenamiento, pero esa solución suele sobreajustar. Se puede limitar mediante:

1. Profundidad máxima: cantidad de reglas sucesivas.
2. Tamaño mínimo de nodo para permitir una nueva división.
3. Mejora mínima de pureza exigida para aceptar un corte.

Estos criterios están relacionados; la profundidad máxima suele ser un buen punto de partida para ajustar hiperparámetros.

Comparación entre K-NN y árboles

- K-NN puede producir fronteras irregulares y ser muy sensible al entrenamiento cuando k es pequeño.
- Los árboles producen fronteras rectangulares y pueden sobreajustar si crecen sin límite.
- Los árboles no suelen depender de la escala de las variables predictoras.
- K-NN sí depende de la escala porque calcula distancias entre observaciones.

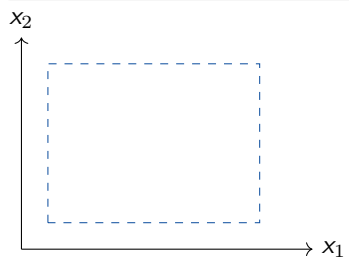
Repaso: K-NN y árboles

1. Explicar cómo K-NN identifica vecinos y genera predicciones.
2. Describir la distancia de Minkowski y las ponderaciones posibles.
3. Explicar las divisiones binarias recursivas de un árbol.
4. Interpretar la impureza de Gini y la profundidad máxima.
5. Relacionar los hiperparámetros de ambos métodos con sesgo y varianza.

¿Por qué importa la escala?

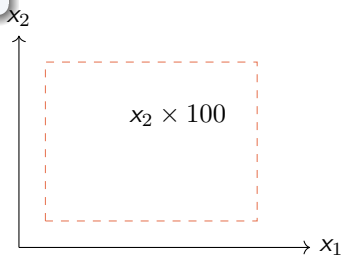
Escalas comparables

Si x_1 y x_2 varían en rangos parecidos, ambas contribuyen de forma comparable a una distancia.



Una escala mucho mayor

Si multiplicamos x_2 por 100, esa característica puede dominar el cálculo de distancia.



Árboles y escala de las características

- Un árbol decide mediante comparaciones de una característica con un umbral, por ejemplo $x_j < c$.
- Cambiar la unidad de medida modifica el valor numérico del umbral, pero mantiene el orden de las observaciones.
- Por ello, árboles basados en estas divisiones son invariantes a cambios de escala monótonos de una característica.
- K-NN, en cambio, combina diferencias de todas las características en una distancia y sí puede verse afectado.

Métodos de reescalado

Método	Transformación por característica
Estandarización	$x_i^* = (x_i - \bar{x})/s$, media 0 y desviación típica 1
Escalado robusto	$x_i^* = (x_i - \text{mediana})/\text{IQR}$, centrado en mediana y rango intercuartílico
Min-Max	$x_i^* = (x_i - \text{mín}(x))/(\text{máx}(x) - \text{mín}(x))$, rango $[0, 1]$
Máximo absoluto	$x_i^* = x_i/\,\text{máx}(\lvert x\rvert)$, rango $[-1, 1]$ y ceros preservados

- El escalado robusto puede convenir si hay valores extremos.
- Máximo absoluto resulta útil con datos dispersos donde los ceros deben conservarse.

Escalado y normalización no son lo mismo

Reescalado

Modifica unidades y rangos; por sí solo conserva la forma de la distribución.

- Estandarización
- Robusto
- Min-Max
- Máximo absoluto

Normalización de la distribución

Transforma la forma para reducir asimetría o el efecto de valores extremos.

- Logaritmo
- Box-Cox
- Mapeo por cuantiles

Transformaciones para reducir asimetría

1. **Logarítmica**: aplica logaritmo a valores positivos; comprime la cola derecha.
2. **Box-Cox**: para $x_i > 0$, $x_i^* = (x_i^\lambda - 1)/\lambda$ si $\lambda \neq 0$; el caso $\lambda = 0$ corresponde al logaritmo.
3. **Mapeo por cuantiles**: transforma cuantiles observados a cuantiles de una distribución objetivo, por ejemplo uniforme $U(0, 1)$.

Precaución

La transformación se ajusta con los datos de entrenamiento y después se aplica a validación, prueba o producción con los mismos parámetros.

Codificación one-hot

Muchos algoritmos necesitan que las variables categóricas se representen numéricamente. La codificación one-hot crea una columna por categoría.

Estado	Iowa	Otro estado
Iowa	1	0
Illinois	0	1
Minnesota	0	1

Algunos modelos, como la regresión lineal con intercepto, pueden beneficiarse de retirar una columna de referencia para evitar redundancia lineal. Los ceros en las demás columnas identifican esa categoría.

Guía práctica de preprocesamiento

- Identificar métodos sensibles a escala, como K-NN.
- Explorar distribuciones y localizar asimetría marcada o valores extremos.
- Reescalar cuando el algoritmo o las unidades lo requieran.
- Representar las categorías con one-hot cuando el modelo no admita categorías directamente.
- Ajustar todas las transformaciones solo con el conjunto de entrenamiento para evitar fuga de información.
- Comparar los pasos de preprocesamiento mediante validación cruzada.

Repaso: preparación de datos

1. ¿Por qué el reescalado importa para K-NN y normalmente no para árboles?
2. ¿En qué se diferencian reescalado y normalización de la distribución?
3. ¿Cómo se representa una variable categórica mediante one-hot?
4. ¿Por qué se ajustan las transformaciones dentro de cada partición de entrenamiento?

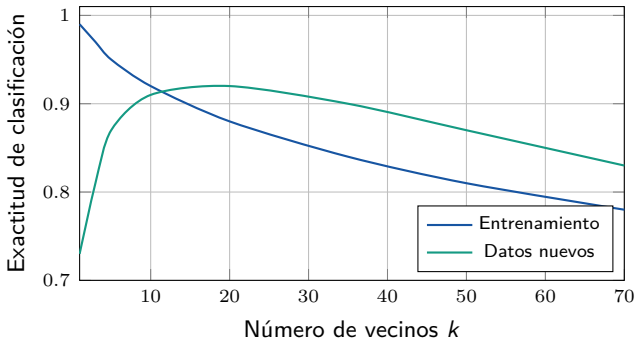
Punto de partida: datos de ejemplo

- Hasta ahora hemos trabajado con datos sintéticos para clasificar muestras como **sanas** o **no sanas**.
- La exactitud sobre los datos de entrenamiento no es nuestro objetivo principal.
- Nos interesa saber cómo funcionará el modelo con **datos nuevos**, que no participaron en el ajuste.

Pregunta guía

¿Cómo estimamos el rendimiento futuro sin usar repetidamente el conjunto de prueba para tomar decisiones?

El valor de k cambia el comportamiento de K-NN



Curvas esquemáticas para explicar la tendencia; no son resultados experimentales del PDF.

- Con valores pequeños de k, K-NN puede ajustarse demasiado al entrenamiento: **sobreajuste**.
- Con valores grandes, la frontera se suaviza en exceso: aumenta el **sesgo**.
- Elegir k exige una estimación fiable del rendimiento en datos no vistos.

Datos de prueba y selección del modelo

- Después de evaluar una vez con el conjunto de prueba, ese conjunto deja de ser una medida imparcial si seguimos tomando decisiones a partir de él.
- Podríamos escoger el k que mejor resultado dé en prueba, pero estaríamos seleccionándolo a posteriori.
- Ese resultado tendería a ser demasiado optimista: hemos elegido el modelo que mejor se ajusta también a las particularidades de esa muestra.

Pregunta

¿Cómo comparamos modelos sin depender repetidamente del conjunto de prueba?

Una partición de validación



- El conjunto de entrenamiento ajusta los parámetros del modelo.
- La validación ayuda a elegir opciones de modelado.
- La prueba se reserva para una evaluación final.

Esquema adaptado; la diapositiva original atribuye la ilustración de partición a algotrading101.com.

Limitaciones de una sola validación

Separar una única muestra de validación del entrenamiento tiene dos inconvenientes:

1. La estimación depende mucho de qué observaciones hayan quedado en validación. Otra partición puede dar un resultado distinto: la estimación tiene **alta varianza**.
2. Podemos acabar ajustando decisiones a esa única validación y perder parte de su utilidad, igual que ocurre al reutilizar la prueba.

Alternativa

Repetir el proceso con distintas particiones de los datos de entrenamiento.

Validación cruzada de k particiones

La validación cruzada repite el ciclo de entrenamiento y validación con diferentes particiones o **pliegues** (*folds*). En cada ronda, un pliegue valida y los demás entrenan.



- En 5 pliegues, cada observación se valida una vez y participa en entrenamiento en las otras cuatro rondas.
- Se resumen los resultados de las cinco rondas para obtener una estimación más robusta.

Algoritmo de validación cruzada de 5 pliegues

```
# Asignar las n observaciones a k pliegues
pliegue = asignar_pliegues(n, k)
predicciones = vacio(n)

# Cada pliegue se usa una vez para validar
for i in range(k):
    train_X = X[pliegue != i]
    train_y = y[pliegue != i]
    valid_X = X[pliegue == i]
    valid_y = y[pliegue == i]

    modelo.fit(train_X, train_y)
    predicciones[pliegue == i] = modelo.predict(valid_X)

# Medir rendimiento con todas las predicciones fuera de pliegue
resultado = metricas(y, predicciones)
```

Idea clave

Cada predicción se obtiene de un modelo que no entrenó con esa observación.

Validación cruzada: dos variantes

La validación cruzada reúne métodos que repiten particiones de los datos para evaluar con observaciones no usadas en cada ajuste.

k-fold

Divide los datos de entrenamiento en k pliegues. En cada ronda se reserva uno y se entrena con los otros $k - 1$.

Es un método **no exhaustivo**: no explora todas las particiones posibles y el resultado puede cambiar según la asignación.

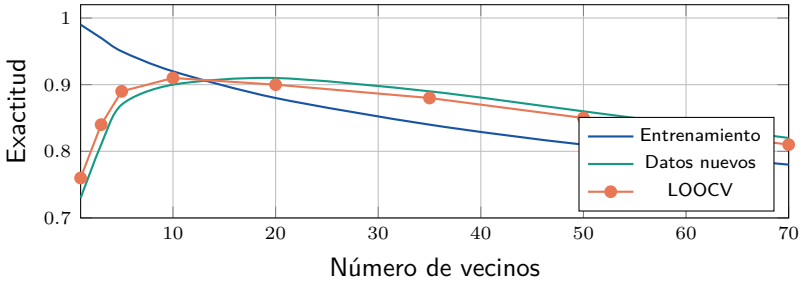
Leave-One-Out (LOOCV)

Caso extremo: cada ronda reserva una sola observación y entrena con las otras $n - 1$. Se repite n veces.

Es exhaustivo respecto a qué observación se deja fuera, pero puede exigir mucho cómputo.

Ejemplo: LOOCV para elegir k

La validación cruzada permite comparar valores de k sin usar el conjunto de prueba durante la selección.



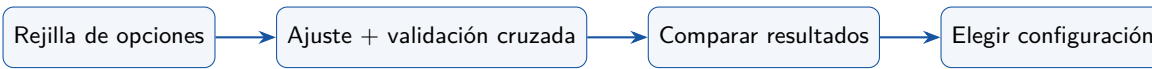
Curvas esquemáticas para ilustrar la comparación; no reproducen mediciones del PDF.

¿k-fold o LOOCV?

- LOOCV es un caso particular de k-fold con $k = n$: cada pliegue contiene una observación.
- Frente al k-fold habitual, LOOCV puede tener **mayor varianza**: las estimaciones cambian más entre muestras.
- Los conjuntos de entrenamiento de LOOCV se solapan casi por completo; por ello, sus errores están muy correlacionados.
- Un k-fold con menos pliegues suele ser más eficiente y puede generalizar mejor en la práctica.
- LOOCV puede resultar útil con muestras pequeñas, aunque repetir el ajuste n veces puede ser costoso.

Búsqueda en rejilla

- La validación cruzada estima el rendimiento usando solo los datos de entrenamiento.
- Por sí sola no decide qué hiperparámetros conviene usar.
- La búsqueda en rejilla evalúa de forma sistemática combinaciones predefinidas de valores.
- Se combina con validación cruzada para comparar las combinaciones con los mismos pliegues.



Ejemplo de rejilla para K-NN

Se exploran $k \in \{3, 4, 5\}$, dos distancias y dos formas de ponderar vecinos:

k	Distancia	Ponderación
3	Euclídea	Uniforme
4	Euclídea	Uniforme
5	Euclídea	Uniforme
3	Manhattan	Uniforme
4	Manhattan	Uniforme
5	Manhattan	Uniforme
3	Euclídea	Por distancia
4	Euclídea	Por distancia
5	Euclídea	Por distancia
3	Manhattan	Por distancia
4	Manhattan	Por distancia
5	Manhattan	Por distancia

- Son $3 \times 2 \times 2 = 12$ configuraciones.
- Cada fila se evalúa con validación cruzada; conviene mantener las mismas particiones para que la comparación sea justa.

Búsqueda aleatoria

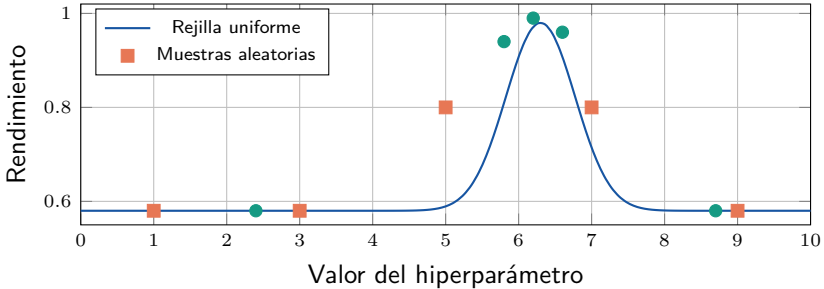
- Una rejilla amplia puede requerir muchas evaluaciones cuando se ajustan varios hiperparámetros.
- La búsqueda aleatoria define distribuciones o rangos y toma una cantidad limitada de muestras de ellos.
- Así puede explorar espacios grandes con un presupuesto de cómputo fijado.
- Para K-NN, por ejemplo, se podría muestrear k de una distribución de Poisson con media $\mu = \sqrt{n}$, donde n es el número de observaciones.

Decisión práctica

La búsqueda aleatoria no garantiza encontrar el óptimo, pero puede cubrir más valores distintos con menos evaluaciones.

Rejilla frente a búsqueda aleatoria

El espaciado uniforme de una rejilla puede pasar por alto valores óptimos cuando el rendimiento cambia mucho en una zona estrecha.



Ejemplo esquemático: los puntos solo ilustran cómo el espaciado afecta la exploración.

Otras estrategias de búsqueda

- Se pueden combinar búsqueda aleatoria y búsqueda en rejilla para explorar espacios grandes con mayor eficiencia.
- Por ejemplo, primero se exploran valores al azar y se descartan zonas con resultados bajos; después se hace una búsqueda más detallada en la región prometedora.
- La **búsqueda por reducción sucesiva** (*successive halving*) también está disponible en bibliotecas como scikit-learn.
- En cada ronda, solo las configuraciones con mejor puntuación reciben más recursos y pasan a la ronda siguiente.
- Es una alternativa que se puede investigar para el proyecto final.

Ideas clave para el próximo cuestionario

1. Dividir los datos de entrenamiento en k partes de tamaño parecido.
2. Entrenar con $k - 1$ partes y predecir sobre la parte que queda fuera.
3. Repetir hasta que cada pliegue haya quedado fuera exactamente una vez.
4. Resumir las métricas de las rondas para estimar el rendimiento.
5. Combinar la validación cruzada con búsqueda en rejilla o aleatoria para elegir hiperparámetros y pasos de preprocesamiento.

Para recordar

El conjunto de prueba se reserva para la evaluación final; no se usa para escoger configuraciones.

Fin

Validación cruzada y selección de modelos

IFCD107 – Especialista en Inteligencia Artificial